MAJOR PROJECT VIVA DEFENSE & TECHNICAL DOCUMENTATION

Automated Log System & Self-Healing Platform

An Enterprise Multi-Tenant Platform with Continuous File Tailing, 26 Pre-Defined Self-Healing Rules, 2-Step OTP Authentication, and Defensive Safety Guardrails

Project Title:	**AutoHeal Cloud Platform**
Architecture:	Strict Model-View-Controller (Java EE Servlet + JDBC + JSP)
2-Step Auth:	Email OTP (Brevo REST API / SecureRandom / 10-Min Invalidation)
Self-Healing Rules:	26 Pre-Defined Production Signatures (DB, Cache, OS, Network)
Target Environments:	Distributed Linux/Windows VMs, AWS EC2, Render, Docker
Security & Auth:	BCrypt Salted Hash, Passwordless OTP, RBAC, Domain API Keys
Guardrails:	Anti-Flapping Rate Limiter & Command Blacklist Sanitizer
Group Size:	4 Engineering Members

Comprehensive Academic Viva Reference Book • 2-Step Auth Deep-Dive • 26 Pre-Defined Error Rules • 28+ Verified Q&A

1. Project Overview & Problem Statement

Modern cloud-native and distributed enterprise software systems produce massive volumes of runtime log data. When critical infrastructure exceptions occur—such as database connection pool exhaustion, memory leakage, stale file locks, or network timeouts—systems suffer severe downtime.

Under traditional operational setups, resolving these issues requires manual intervention: an on-call engineer is alerted, logs into the remote server via SSH, inspects log files, identifies the failure signature, and manually runs corrective scripts. This manual turnaround can take 30 to 120 minutes.

The **Automated Log System & Self-Healing Platform (AutoHeal)** provides an autonomous, closed-loop solution that continuously tails logs at the edge, parses high-severity errors, matches them against pre-configured remediation rules, and executes corrective actions in real time—reducing Mean Time to Resolution (MTTR) from hours to **less than 5 seconds**.

2. Two-Step Authentication (2FA / OTP Verification)

To ensure enterprise-grade security and prevent unauthorized access to sensitive remediation infrastructure, AutoHeal implements a robust **Two-Step Authentication / Passwordless OTP Verification Subsystem**:

2.1 Technologies & Libraries Used

- **Email Gateway API: Brevo (formerly Sendinblue) HTTP REST API v3** (`https://api.brevo.com/v3/smtp/email`). Using direct HTTPS API calls circumvents cloud port blocks (e.g. cloud providers blocking standard SMTP port 25/587) and guarantees instantaneous email delivery.
- **Secure Cryptographic Generator:** `java.security.SecureRandom` in `OTPUtil.java` generates non-deterministic, cryptographically strong 4-digit numeric OTP codes ranging from `1000` to `9999`.
- **Asynchronous Non-Blocking Dispatch:** `CompletableFuture.runAsync()` in `EmailService.java` offloads email network latency from the HTTP servlet thread, giving users sub-second response times on the UI.
- **Database Model & Invalidation:** `otp_verifications` table tracking `email`, `otp_code`, `expires_at`, and `is_used`.

2.2 Step-by-Step 2-Step Authentication Lifecycle

```
[ User enters Email on Web UI ]
             |
             v
[ OTPServlet (/api/otp/send) ]
             |
             +---> 1. Check if user exists & verify role in DB (UserDAO)
             |
             +---> 2. SecureRandom generates OTP code: e.g. "8492" (OTPUtil)
             |
             +---> 3. Insert record into `otp_verifications` with 10-minute expiry (OTPDAO)
             |
             +---> 4. Asynchronously call Brevo HTTP API to dispatch email (EmailService)
             |
             v
[ User receives Email and inputs Code into UI ]
             |
             v
[ OTPServlet (/api/otp/verify) ]
             |
             +---> 1. Query DB for matching email + otp_code where is_used = FALSE
             +---> 2. Validate current timestamp < expires_at
             +---> 3. Mark is_used = TRUE (Single-Use Invalidation to prevent replay attacks)
             +---> 4. Bind User Principal into HTTPSession & redirect to Dashboard
```

2.3 Dual Login Strategy

User Persona	Authentication Flow	Security Defense
Organization Owner	Email + Master Password	**BCrypt Salted Hash** (10 rounds) stored in `users.password_hash`, protecting against rainbow table and brute-force attacks.
Employees & Developers	Passwordless 2-Step OTP Authentication	Email verification code delivered via **Brevo REST API**, valid strictly for 10 minutes, single-use enforced via database flag.

3. Pre-Defined Errors That Get Automatically Fixed

The AutoHeal database is pre-seeded with **26 battle-tested infrastructure error signatures** categorized across 4 automated action types:

#	Category	Error Signature (Pattern)	Action Type	Automated Remediation Script
1	**Database**	`Connection pool exhausted`	`RESET_CONNECTION`	`echo "Resetting DB Connections"`
2	**Memory**	`memory leak`	`CLEAR_CACHE`	`echo "Clearing cache & freeing memory"`
3	**Cache**	`RedisCacheException`	`CLEAR_CACHE`	`scripts/flush-redis-cache.sh`
4	**Service Freeze**	`freeze`	`RESTART_SERVICE`	`npm restart`
5	**CPU Overload**	`cpu lag`	`RESTART_SERVICE`	`pm2 restart app`
6	**Storage/Disk**	`No space left on device`	`CUSTOM_SCRIPT`	`sh scripts/clear-disk-cache.sh`
7	**Gateway / Proxy**	`502 Bad Gateway`	`RESTART_SERVICE`	`pm2 restart backend-worker`
8	**Lockfiles**	`EEXIST`	`CUSTOM_SCRIPT`	`rm -f /tmp/*.lock /var/run/*.pid`
9	**OS Limits**	`Too many open files`	`CUSTOM_SCRIPT`	`ulimit -n 65535 && pm2 reload all`
10	**Process State**	`Defunct`	`CUSTOM_SCRIPT`	`pkill -9 -f defunct`
11	**SSL / TLS**	`Certificate Expired`	`CUSTOM_SCRIPT`	`certbot renew && systemctl reload nginx`
12	**Thread Freeze**	`ServerThreadFrozen`	`RESTART_SERVICE`	`npm restart`
13	**Thread Freeze**	`frozen`	`RESTART_SERVICE`	`npm restart`
14	**Storage/Disk**	`ENOSPC`	`CUSTOM_SCRIPT`	`sh scripts/clear-disk-cache.sh`
15	**Storage/Disk**	`STORAGE_CRITICAL`	`CUSTOM_SCRIPT`	`sh scripts/clear-disk-cache.sh`
16	**OS Limits**	`EMFILE`	`CUSTOM_SCRIPT`	`ulimit -n 65535 && pm2 reload all`
17	**OS Limits**	`OS_RESOURCE_LIMIT`	`CUSTOM_SCRIPT`	`ulimit -n 65535 && pm2 reload all`
18	**Filesystem**	`FILESYSTEM_CONFLICT`	`CUSTOM_SCRIPT`	`rm -f /tmp/*.lock /var/run/*.pid`
19	**SSL / TLS**	`CERT_HAS_EXPIRED`	`CUSTOM_SCRIPT`	`certbot renew && systemctl reload nginx`
20	**SSL / TLS**	`certificate has expired`	`CUSTOM_SCRIPT`	`certbot renew && systemctl reload nginx`
21	**SSL / TLS**	`TLSError`	`CUSTOM_SCRIPT`	`certbot renew && systemctl reload nginx`
22	**Zombie Proc**	`ProcessZombieException`	`CUSTOM_SCRIPT`	`pkill -9 -f defunct`
23	**Gateway / Proxy**	`GATEWAY_ERROR`	`RESTART_SERVICE`	`pm2 restart backend-worker`
24	**CPU Overload**	`CPUStarvationException`	`RESTART_SERVICE`	`pm2 restart app`
25	**CPU Overload**	`CPU computation`	`RESTART_SERVICE`	`pm2 restart app`
26	**Service Freeze**	`ServerFreezeException`	`RESTART_SERVICE`	`npm restart`

4. Project Role Division (4 Team Members)

In academic project vivas, examiners evaluate technical ownership and individual contribution across modules:

Member 1: System Architect & Agent Core Specialist (Backend Lead)

Agent Daemon & Ingestion

Primary Focus: Client-side continuous log monitoring, persistent daemon architecture, and high-performance ingestion pipeline.

- **Modules Owned:** `LogAgent.java`, `LogIngestionServlet.java`, `DBConnection.java`, `LogDAO.java`, `schema.sql`.
- **Key Technical Contributions:**
  - Engineered the log-tailing engine in `LogAgent` using `RandomAccessFile.seek()`, maintaining byte-offset pointers.
  - Implemented file truncation and log rotation detection logic (resetting pointer to zero when `fileLength < lastPosition`).
  - Built the multi-line stack trace aggregator using regex `(?i)(ERROR|CRITICAL|EXCEPTION|FATAL)` and indentation detection.
  - Developed the local command execution sub-routine using Java `ProcessBuilder` supporting both Windows (`cmd.exe /c`) and Linux (`sh -c`).
- **Viva Pitch:** "*I architected the distributed log ingestion pipeline and engineered the standalone LogAgent daemon that tails application logs in real-time with zero overhead and runs corrective remediation scripts returned by the server.*"

Member 2: Rule Engine & Remediation Dispatch Engineer

Rule Engine & DevOps

Primary Focus: Pattern-matching algorithm, 26 pre-defined error rules, action dispatching, and VCS/GitHub integration.

- **Modules Owned:** `AutoHealingRuleDAO.java`, `AutoHealingRuleServlet.java`, `GitHubIntegrationService.java`, `RuleConversionService.java`.
- **Key Technical Contributions:**
  - Designed the pattern-matching rule engine that evaluates incoming log messages and stack traces against the 26 pre-defined rules.
  - Structured action classifications: `RESTART_SERVICE`, `CLEAR_CACHE`, `RESET_CONNECTION`, and `CUSTOM_SCRIPT`.
  - Engineered the rule management CRUD subsystem allowing administrators to toggle rules active/inactive and assign custom scripts.
  - Built the GitHub integration pipeline (`GitHubIntegrationService`) automating hotfix branch creation, patch commits, and Pull Requests.
- **Viva Pitch:** "*I designed the pattern-matching rule engine that evaluates incident signatures against active remediation rules in milliseconds, and built the action dispatch subsystem and GitHub hotfix pipeline.*"

Member 3: Security, Guardrails & 2FA Compliance Specialist

Safety Guardrails & 2FA

Primary Focus: Two-Step OTP Authentication, safety guardrails, anti-flapping protection, multi-tenant RBAC, and audit compliance.

- **Modules Owned:** `OTPServlet.java`, `OTPDAO.java`, `OTPUtil.java`, `EmailService.java`, `RateLimiter.java`, `CommandSanitizer.java`, `AuthFilter.java`, `RBACFilter.java`, `PasswordUtil.java`, `AuditLogDAO.java`.
- **Key Technical Contributions:**
  - Implemented the 2-Step OTP Authentication pipeline integrating Brevo REST API, `SecureRandom`, and 10-minute database token expiration.
  - Implemented the sliding-window `RateLimiter` using thread-safe `ConcurrentHashMap` to prevent catastrophic infinite healing loops (>3 fixes in 15 mins).
  - Developed `CommandSanitizer` to inspect scripts against blacklisted commands (blocking `rm -rf`, `mkfs`, `dd`, `shutdown`, `reboot`).
  - Engineered the multi-tenant Role-Based Access Control hierarchy (`OWNER`, `MANAGER`, `SENIOR_DEVELOPER`, `DEVELOPER`).
- **Viva Pitch:** "*I built the 2-Step OTP email authentication pipeline, platform safety guardrails preventing infinite restart loops and destructive command injections, and the enterprise RBAC infrastructure.*"

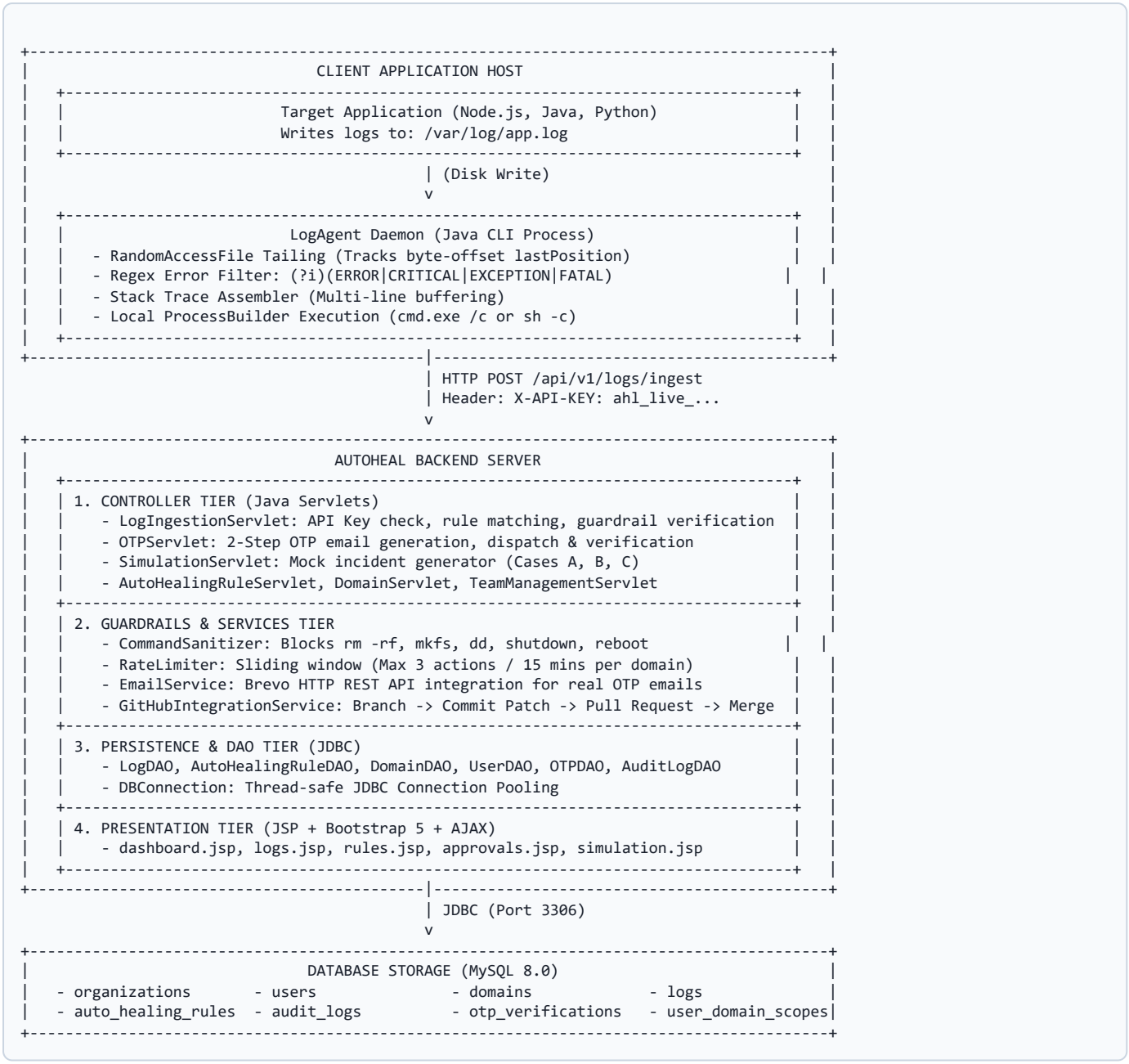
Member 4: Full-Stack Web Interface & Simulation/DevOps Engineer

Frontend & Simulation

Primary Focus: Web UI dashboards, real-time log monitoring views, approval workflows, failure simulation testbed, and deployment.

- **Modules Owned:** `SimulationServlet.java`, `dashboard.jsp`, `logs.jsp`, `rules.jsp`, `approvals.jsp`, `simulation.jsp`, `Dockerfile`.
- **Key Technical Contributions:**
  - Built the responsive operational web dashboard using Bootstrap 5, custom CSS, and JSP displaying real-time system metrics, recent incidents, and status badges.
  - Created the Interactive Simulation Testbed (`SimulationServlet`) enabling one-click reproduction of Known Errors (Case A), Unmatched Errors (Case B), and Loop Lockout (Case C).
  - Engineered domain management interfaces with API Key generation, mask toggling, and user-to-domain scoping.
  - Containerized the platform with Docker and configured the embedded Jetty server for local and cloud deployment.
- **Viva Pitch:** "*I developed the operational web dashboard and built the interactive failure simulation engine that allows us to demonstrate and test all self-healing workflows and safety guardrails live in front of the examiners.*"

5. System Architecture & Coding Logic



5.1 2-Step Authentication Code: Generating & Dispatching OTP

```
// OTPServlet.java: Generating and dispatching OTP asynchronously via Brevo API
String otpCode = OTPUtil.generate4DigitOTP(); // SecureRandom 1000..9999
otpDAO.saveOTP(email, otpCode, 10);          // Persist to DB with 10-minute expiry
EmailService.sendOTPEmailAsync(email, otpCode); // Asynchronous HTTP POST to Brevo API
```

5.2 2-Step Authentication Code: Validating & Invalidating OTP

```
// OTPServlet.java: Single-Use OTP Validation
boolean isValid = otpDAO.validateOTP(email, otpCode);
if (isValid) {
    otpDAO.markAsUsed(email, otpCode); // Invalidate token immediately to prevent replay attacks
    User user = userDAO.findByEmail(email);
    HttpSession session = req.getSession(true);
    session.setAttribute("user", user); // Initialize authenticated session
    JSONUtil.sendJsonResponse(resp, 200, true, "OTP Verified successfully.", null);
}
```

5.3 Edge Continuous File Tailing (`LogAgent.java`)

```
long fileLength = file.length();
if (fileLength < lastPosition) {
    lastPosition = 0; // File was rotated or truncated
} else if (fileLength > lastPosition) {
    try (RandomAccessFile raf = new RandomAccessFile(file, "r")) {
        raf.seek(lastPosition); // Direct jump to newly written bytes
        String line;
        while ((line = raf.readLine()) != null) {
            newLines.add(new String(line.getBytes(StandardCharsets.ISO_8859_1), StandardCharsets.UTF_8));
        }
        lastPosition = raf.getFilePointer();
        processLines(newLines);
    }
}
```

6. Comprehensive Viva Questions & Answers

Category A: Two-Step Authentication & Platform Security

> **Q1: What did you use for Two-Step Authentication (2FA), and how is it implemented?**
> **Answer:** We implemented an Email-based Two-Factor / Passwordless OTP Verification subsystem. It uses `java.security.SecureRandom` in `OTPUtil.java` to generate a cryptographically strong 4-digit token. The code is saved to the `otp_verifications` database table with a 10-minute expiration window and dispatched asynchronously via the **Brevo (Sendinblue) HTTP REST API v3** in `EmailService.java`. Once submitted, the token is verified and marked as `is_used = TRUE` to prevent replay attacks.

> **Q2: Why use the Brevo HTTP API instead of standard JavaMail SMTP for OTP emails?**
> **Answer:** Standard SMTP (ports 25, 465, 587) is frequently blocked or rate-limited by cloud hosts (like Render, AWS EC2, or institutional firewalls) to prevent spam. By utilizing Brevo's direct HTTPS REST API (Port 443), our requests bypass port blocks, execute with higher delivery reliability, and transmit in under 2 seconds.

> **Q3: How do you prevent OTP replay attacks or brute-force guessing?**
> **Answer:** 1) **Single-Use Invalidation:** As soon as an OTP is verified in `OTPServlet`, `otpDAO.markAsUsed()` sets `is_used = TRUE`, preventing the same code from ever being reused; 2) **Time-to-Live (TTL):** The database query enforces `expires_at > CURRENT_TIMESTAMP`, rejecting expired codes; 3) **Cryptographic Randomness:** We use `SecureRandom` instead of standard `java.util.Random`, ensuring tokens cannot be predicted via seed analysis.

> **Q4: What is the difference between the Owner login and the Employee login?**
> **Answer:** - **Owner Login:** Uses master password authentication protected by **BCrypt salted hashing** (10 rounds), intended for root administrators. - **Employee / Developer Login:** Uses **Passwordless 2-Step OTP Authentication**. The employee enters their registered email, receives a one-time code in their inbox, and logs in without needing to remember a static password.

> **Q5: How does the system authenticate the client LogAgent sending logs?**
> **Answer:** When an organization registers a domain, a 64-character UUID token prefixed with `ahl_live_` is generated. The agent transmits this token in the `X-API-KEY` HTTP header. `LogIngestionServlet` validates this key against the `domains` database table before processing any logs.

Category B: Pre-Defined Error Rules & Healing Logic

> **Q6: Which pre-defined errors does your system automatically fix?**
> **Answer:** Our system is pre-configured with **26 infrastructure error rules** across 4 core action categories:
> - **Database:** "*Connection pool exhausted*" → `RESET_CONNECTION`.
> - **Memory & Cache:** "*memory leak*", "*RedisCacheException*" → `CLEAR_CACHE` (flushing redis/memory).
> - **Freezes & Starvation:** "*freeze*", "*frozen*", "*ServerFreezeException*" → `npm restart`; "*cpu lag*", "*CPUStarvationException*" → `pm2 restart app`.
> - **Network & Gateways:** "*502 Bad Gateway*", "*GATEWAY_ERROR*" → `pm2 restart backend-worker`.
> - **Disk & Filesystem:** "*No space left on device*", "*ENOSPC*", "*STORAGE_CRITICAL*" → `clear-disk-cache.sh`; "*EEXIST*", "*FILESYSTEM_CONFLICT*" → cleans lockfiles (`rm -f /tmp/*.lock`).
> - **OS Limits:** "*Too many open files*", "*EMFILE*", "*OS_RESOURCE_LIMIT*" → sets `ulimit -n 65535` and reloads processes.
> - **Zombie Processes:** "*Defunct*", "*ProcessZombieException*" → executes `pkill -9 -f defunct`.
> - **SSL/TLS Certificates:** "*Certificate Expired*", "*TLSError*" → executes `certbot renew && systemctl reload nginx`.

> **Q7: How does the pattern matching rule engine prioritize rules?**
> **Answer:** `AutoHealingRuleDAO` queries active rules matching the specific `domain_id` as well as global rules (where `domain_id IS NULL`). The engine performs a case-insensitive substring search on both the log message and stack trace. The first matching active rule is executed, and its details are returned to the agent.

> **Q8: What happens when an error occurs that doesn't match any of the 26 pre-defined rules?**
> **Answer:** The log entry is persisted in the database with status `PENDING`. It appears immediately in the operational dashboard. An administrator or manager can review the incident, resolve it manually, or click to create a brand-new auto-healing rule so future occurrences are resolved autonomously.

Category C: Edge LogAgent & Ingestion Mechanics

Q9: How does the LogAgent tail log files without causing high CPU spikes?
Answer: The agent uses `RandomAccessFile` with an internal byte-offset pointer (`lastPosition`). Rather than reading the entire file repeatedly, it seeks directly to `lastPosition`, reads only newly written bytes, and sleeps for a configurable polling interval (1000ms), maintaining near-zero CPU and memory usage.

Q10: What happens when a log file is rotated (logrotate) or truncated?
Answer: The agent checks `file.length()` against `lastPosition`. If `file.length() < lastPosition`, it detects that log rotation has occurred, safely resets `lastPosition = 0`, and resumes tailing from the start of the new file without crashing or losing data.

Q11: How does the agent handle multi-line stack traces?
Answer: It uses stateful line buffering. When a line matches regex `(?i)(ERROR|CRITICAL|EXCEPTION|FATAL)`, an error block opens. Subsequent lines beginning with whitespace, tabs, or `at` are accumulated into the stack trace buffer until a normal line or end of chunk signals the end of the error.

Q12: How does the agent execute the healing script locally?
Answer: When the central server responds with a `commands` JSON array, the agent invokes `ProcessBuilder`. It detects the OS (running `cmd.exe /c` on Windows or `sh -c` on Linux), executes the script, and logs the process exit code.

Category D: Safety Guardrails (Flapping & Injection Defense)

Q13: What is flapping, and how does your RateLimiter prevent infinite restart loops?
Answer: Flapping happens when an underlying issue (e.g. corrupted database) causes a service to crash immediately upon restart. An automated restart rule would trigger endlessly in an infinite loop, exhausting CPU and disk. Our `RateLimiter` maintains execution timestamps in a 15-minute sliding window; if more than 3 executions occur in 15 minutes, it halts automation and marks the log as `LOOP_DETECTED`.

Q14: How does the CommandSanitizer protect systems from destructive commands?
Answer: Before any script is sent to an agent, `CommandSanitizer.isSafe()` inspects the command against a blacklist containing destructive patterns: `rm -rf`, `mkfs`, `dd `, `shutdown`, `format`, `reboot`, and `> /dev/sda`. If a match is found, execution is blocked and marked `SECURITY_BLOCKED`.

Category E: Database & System Design

Q15: How is multi-tenancy enforced in your database design?
Answer: The root table is `organizations`. All domains, users, rules, and log entries maintain foreign keys referencing `organization_id`. Servlets filter data by the authenticated session's organization to ensure strict data isolation.

Q16: How do you prevent SQL Injection across all DAO classes?
Answer: All database queries across all DAO classes (`LogDAO`, `AutoHealingRuleDAO`, `OTPDAO`, etc.) strictly use parameterized `PreparedStatement` objects. String concatenation in SQL statements is completely forbidden.

Q17: How does the system handle time zone differences for live streaming logs across global cloud servers?
Answer: Server and agent logs are stored in UTC / epoch milliseconds in MySQL to preserve an absolute single source of truth regardless of server geographical location. When displayed on the web console and dashboards, the platform standardizes and dynamically localizes all log timestamps to **Indian Standard Time (IST • Asia/Kolkata • UTC+05:30)** using JSTL `<fmt:timeZone value="Asia/Kolkata">` and JavaScript's `Intl.DateTimeFormat` with target timezone `Asia/Kolkata`. This guarantees consistent, precise operational timestamps for all live streams and incident alerts.

7. Viva Live Demonstration Walkthrough

To demonstrate the platform live to examiners, use the built-in **Simulation Engine** (`/simulation`). This demonstrates all core features without needing external server failures:

Scenario	Trigger Action	System Reaction & What to Highlight to Examiners
Demo 1: Known Error (Auto-Heal)	Click "*Trigger Case A: Known Error*" on the Simulation tab.	• Simulates: "*Connection pool exhausted: Timeout waiting for idle database connection*". • The rule engine matches Rule #1 (`RESET_CONNECTION`). • Status transitions to **AUTO_HEALED** in <10ms. • Show the command payload returned and the audit log entry created.
Demo 2: Unmatched Error (Manual Triage)	Click "*Trigger Case B: Unknown Error*" on the Simulation tab.	• Simulates: "*NullPointerException in PaymentsGateway processing transaction*". • No active rule matches the pattern; status is recorded as **PENDING**. • Show the error appearing in the dashboard for manual administrative review. • Demonstrate creating a new rule for this pattern so future occurrences are auto-healed.
Demo 3: Flapping Guardrail (Rate Limiter Lockout)	Click "*Trigger Case C: Loop Lockout*" on the Simulation tab.	• Fires 4 rapid consecutive database failure logs within milliseconds. • Executions 1, 2, and 3 are healed normally. • On the 4th attempt, the sliding-window `RateLimiter` triggers a lockout. • Status becomes **LOOP_DETECTED** with warning: "*Auto-fix loop detected (>3 times in 15m)*". • Explain how this defensive guardrail saves servers from infinite restart loops.

Summary Checklist for Viva Examiners

> **Key Strengths to Emphasize During Defense:**
> 1. **Full-Stack Production MVC:** Native Java Servlets, JDBC DAOs, JSP front-end, zero heavy framework bloat.
> 2. **2-Step Authentication:** Passwordless 10-minute OTP verification using Brevo REST API and SecureRandom.
> 3. **26 Pre-Defined Self-Healing Rules:** Comprehensive signatures for DB connection resets, cache flushes, process reloads, lock removals, and SSL renewals.
> 4. **Safety First:** Defensive engineering through sliding-window rate limiters and command blacklist sanitizers.
> 5. **Edge Daemon:** Standalone background agent with byte-offset file pointer tailing and multi-line stack trace stitching.